

Woodshop RFID / Login-Server design (current)

Status: actively evolving design discussion with Bruce. Last updated 2026-08-19. Paused here pending an architecture/implementation meeting — no code written yet. Supersedes the original single-Pi, single-user-database design in `app.py` / `master_server.py` / `server_card_writer.py` as uploaded.

System overview

Two physically separate machines, connected via Ethernet, plus one ESP32 node per shop machine (unchanged from original design):

- **Server** (Bruce's existing Pi): runs the machine-access TCP listener (port 35487, largely unchanged) and a small web admin app. Writes the "access" section of member RFID cards. No longer holds a full member database, no longer has WWVB/DS3231 hardware — has NO independent time source at all now (see Synced clock, below).
- **Login** (Lee's Raspberry Pi 5): a standalone entry/exit terminal at the shop door. Has its own WWVB receiver + DS3231 RTC (this hardware moved here from Server, and is now the system's sole authoritative time source). Writes the "identity" section of member RFID cards, and separately runs the day-to-day login/logout reader.
- **Machine nodes** (ESP32, one per machine): read both card sections at card-insert, verify both Ed25519 signatures, cross-check memberID matches between the two sections, and talk to Server over the existing 16-byte binary protocol (port 35487) to get an authorization decision.

RFID card format (NTAG215)

Two independently Ed25519-signed sections, disjoint page ranges, each writable without touching or invalidating the other:

Access section (Server/Bruce writes, Bruce's Ed25519 key):

- `card_type(1) + version(1) + member_id(4, uint32) + permissions bitmap (32 bytes = 256 bits = 256 machines, 8 x uint32) = 38 bytes payload`
 - 64-byte Ed25519 signature = 102 bytes -> pad to 104 (26 pages)

Identity section (Login/Lee writes, Lee's Ed25519 key):

- `card_type(1) + version(1) + member_id(4) + first_name(16 or 20?) + last_name(16 or 20?)`
[TBD — confirm field width vs. the 20-byte width used in the login network message] = payload
 - 64-byte Ed25519 signature

Both signatures are kept (resolved) — the memberID cross-check performed by machine clients is an *additional* consistency check on top of signature verification, not a replacement for it. Signature verification is what actually prevents a member from self-editing their own access bits with an off-the-

shelf NFC writer; the memberID cross-check only catches sections spliced together from two different physical cards.

Example page allocation (still needs confirming against actual usable page range): pages 4-29 access section, pages 30-55 identity section, 56+ reserved.

Login -> Server network message

Login's Pi sends one message to Server per login event and one per logout event, over Ethernet, to Server at 192.168.0.5 port 45432.

Fixed 54-byte message, proposed struct format >BQBI20s20s (big-endian, matching the style of the existing 16-byte machine protocol in master_server.py):

messageType	1 byte (uint8)	0 = login, 1 = logout
timestamp	8 bytes (uint64)	Login's wwvb/DS3231-sourced time
memberStatus	1 byte (uint8)	eligible / ineligible (e.g. dues status)
memberID	4 bytes (uint32)	
firstName	20 bytes	
lastName	20 bytes	

Single timestamp field serves two purposes (resolved, confirmed no need for a second field):

1. On a login message, it becomes `active_members.login_time` directly (Bruce confirmed the message timestamp is the login time, accurate to within a few seconds — good enough).
2. On either message type, it's fed into Server's SyncedClock (see below).

On receipt, Server:

1. Feeds the timestamp into SyncedClock.
2. Updates the active_members table: a login message inserts/replaces a row (including `login_time` = this message's timestamp), a logout message deletes the row for that member_id.

Server-side synced clock (resolved, then simplified further 2026-08-19)

Server has no independent time source at all anymore (no WWVB, no DS3231). Original plan: one small clock abstraction everything routes through -- anchor on the last Login-message timestamp + `time.monotonic()` at receipt, derive "now" as anchor + monotonic-elapsed-since, re-anchor only on >5s deviation.

Simplified (2026-08-19): Bruce pointed out that since `login_time` is stored directly from the message (not derived from Server's clock) and `remove_time` is computed from `insert_time` + `connect_time` (not resampled from Server's clock either), SyncedClock's *only* remaining job is knowing roughly when midnight has passed, to trigger the active_members reset — and that's not a critical, high-precision requirement. So: update it on every incoming message (login or logout) —

cheap, no downside, no real precision requirement riding on it anymore. Simple implementation is fine; no need to over-engineer the deviation/re-anchor logic for this reduced role.

Server-side data (post-redesign)

Server's old full user database (users.json: name, email, permissions, RFID, expiry, etc.) is gone. What remains on Server:

- **machineID table/file** — unchanged from original design (machine number -> name, etc.)
- **Reduced user/access file** — memberID + 256-bit access bitmap only. Used solely for writing the access section of cards; no longer used for runtime authorization decisions.
- **active_members table** — implement as a real SQLite table in the existing Database class / woodshop.db (Server already uses sqlite3 with commit-per-write, so this fits existing infrastructure rather than adding a new file format). Columns: member_id (PK), status, first_name, last_name, login_time (resolved: yes, add login_time, set directly from the login message's timestamp field).
 - Login message -> INSERT OR REPLACE
 - Logout message -> DELETE WHERE member_id = ?
 - **Persistence strategy (resolved):** write-through (commit on every login/logout event), not periodic snapshotting. Event frequency is low (people walking in the door), so there's no performance reason to batch, and write-through closes the gap a periodic snapshot leaves (a crash between snapshots silently drops whoever badged in during that window). Because SQLite already commits synchronously in the existing code pattern, this happens "for free" by just using the same Database class.
 - **Midnight reset (resolved):** Server deletes all rows from active_members at midnight (using SyncedClock, now much simpler per above — this is its only real job). Login's Pi does not separately send synthetic logout messages for this.
- **log file** — unchanged from original design.

Someone not present in active_members cannot use any machine — this is now the sole gate for machine access at runtime (replacing the old users.json-driven per-user "authorized" lookup). This gives real teeth to the existing-but-previously-unused STATUS_MEMBER_NOT_AUTHORIZED status code in master_server.py's process_message — that path needs to actually check active_members now.

remove_time fix (kept from earlier proposal): compute remove_time = insert_time + connect_time, using the ESP32 node's own measured connect_time value, rather than resampling any Server-side clock. The node's own hardware timer is a more direct measurement than a Server clock estimate, however simple or elaborate.

Effort / scope

- server_card_writer.py (Server): rework to two-section format, 8-word (256-bit) permission bitmap, Bruce's Ed25519 key for the access section only.

- `app.py` (Server): user-management UI shrinks significantly — no more name/email/expiry CRUD, no more CSV of full user records; becomes memberID + permission-bitmap management only, for the purpose of writing cards.
- `master_server.py` (Server): new `active_members` SQLite table + queries; new listener on port 45432 for Login's 54-byte messages; simple `SyncedClock` (now just needs to roughly track midnight, see above); midnight clear job for `active_members`; `process_message` needs to actually consult `active_members` for the `MEMBER_NOT_AUTHORIZED` path; `record_remove` uses `insert_time` + `connect_time`. `wwvb.py` and the DS3231 integration are removed from Server entirely.
- Login (Lee's Pi 5): new codebase, not yet built. Needs: `wwvb.py`-equivalent
 - DS3231 time source (kept from original Server design, now here instead), its own PN532 write tool for the identity card section (own Ed25519 keypair), the day-to-day login/logout card-read loop, and the port-45432 client sending the 54-byte messages to Server.
- ESP32 node firmware (not in hand / not uploaded to this session): verify both Ed25519 signatures (two different public keys), cross-check memberID between sections, widen permission handling from 128 to 256 bits. Exact firmware diff size unknown without seeing that code.

Status / next steps

Design paused here (2026-08-19) — Bruce has an architecture/implementation meeting to discuss this before any code gets written. No implementation started yet.

Open questions

- Exact field list/width for the identity card section (just first/last name + memberID, or does memberStatus also get written to the card itself, redundant with the live `active_members` table?).
- Confirm actual NTAG215 usable page range before finalizing the page allocation table.
- Which component to implement first — `master_server.py` (`SyncedClock` + `active_members` + port 45432 listener) or `server_card_writer.py` (two- section card format)? Not yet decided; on hold pending the meeting.